

A. Computer Arithmetic

A.1. Finite-digit arithmetic

- $x \oplus y := fl(fl(x) + fl(y))$ $x \ominus y := fl(fl(x) - fl(y))$
- $x \otimes y := fl(fl(x) \times fl(y))$ $x \oslash y := fl(fl(x) \div fl(y))$
- Three don'ts:
 - $a + b$ for $|a| \gg |b| > 0$
 - $a - b$ for $a \approx b$
 - $a \times b$ for $0 < |a|, |b| \ll 1$

A.2. Algorithms and pseudocodes

- **Truncation error:** error introduced by truncation of a series
- Use **recurrence relation** to express $(k + 1)$ -th term using k -th term of series
- Bound truncation error: use triangle inequality to distribute absolute value, then geometric series

A.3. Matrix multiplication

General matrix multiplication:

```

1: for  $i = 1, \dots, m$  do
2:   for  $j = 1, \dots, p$  do
3:      $c_{ij} \leftarrow a_{i1}b_{1j}$ 
4:     for  $k = 2, \dots, n$  do
5:        $c_{ij} \leftarrow c_{ij} + a_{ik}b_{kj}$ 
6: return  $C = (c_{ij})_{m \times p}$ 

```

Special matrices:

- **Upper triangular:** $a_{ij} = 0$ if $i > j$ **Lower triangular:** $a_{ij} = 0$ if $i < j$
- **Upper Hessenberg:** $a_{ij} = 0$ if $i > j + 1$ **Lower Hessenberg:** $a_{ij} = 0$ if $i < j - 1$
- **Tridiagonal:** $a_{ij} = 0$ if $|i - j| > 1$

Multiplying two special matrices requires fewer operations. To save computational cost:

1. Identify the zero entries in the matrix product, and skip the computation of these entries
2. For non-zero entries, avoid adding zero terms in the sum $c_{ij} = \sum_{k=1}^n a_{ik}b_{kj}$

Time complexity: $f(n) = O(g(n))$ if there exists a positive constant M and a real number N such that

$$|f(n)| \leq Mg(n) \quad \text{for all } n > N.$$

B. Linear Systems of Equations

B.1. Gaussian elimination

Let r_i be the index of the row with label i . Read $r_i = j$ this as "label i is at the j -th row".

$$\begin{array}{l} 1: \\ 4: \\ 2: \\ 3: \end{array} \left(\begin{array}{cccc|c} 1 & -1 & 2 & -1 & -8 \\ 0 & 0 & -1 & -1 & -4 \\ 0 & 2 & -1 & 1 & 6 \\ 0 & 0 & 3/2 & 9/2 & 13 \end{array} \right) \quad r_1 = 1, r_2 = 3, r_3 = 4, r_4 = 2$$

Pivoting strategies for Gaussian elimination:

- **Partial pivoting:** In pivot column, entry with largest magnitude is chosen as pivot.
- **Scaled partial pivoting:** Before running Gaussian elimination, compute largest absolute value $s_1, \dots, s_n$ in each row. In pivot column, entry with largest relative magnitude is chosen as pivot.

Given augmented matrix after Gaussian elimination,

1. Determine row labels of resulting augmented matrix (start with $1, \dots, n$, swap row labels)
2. Backward substitution to find the solution $x_1, \dots, x_n$
3. Restore the augmented matrix using **inverse** operations

B.2. LU factorization

- Using row i to eliminate row j , **multiplier** $m_{ji} = \frac{a_{ji}}{a_{ii}}$
- If $Ax = b$ can be solved by Gaussian elimination without pivoting, then $A = LU$ for unit lower triangular L , upper triangular U .

$$L = \begin{pmatrix} 1 & & & \\ m_{21} & 1 & & \\ \vdots & \vdots & \ddots & \\ m_{n1} & m_{n2} & \cdots & 1 \end{pmatrix} \quad U = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ & a_{22} & \cdots & a_{2n} \\ & & \ddots & \vdots \\ & & & a_{nn} \end{pmatrix}$$

- Given LU factorisation $A = LU$, we can solve $Ax = b$ as follows:

$$Ax = b \iff LUx = b$$

1. Solve lower triangular system $\tilde{L}b = b$ for $\tilde{b}$ using **forward sub**
 2. Solve upper triangular system $Ux = \tilde{b}$ for x using **backward sub**
- An $n \times n$ **permutation matrix** $P = (p_{ij})$ is obtained by rearranging rows of identity matrix I_n .
 - With partial pivoting, $PA = LU$ for some permutation matrix P , unit lower triangular L , upper triangular U .
 - Given LU factorization $PA = LU$, we solve $Ax = b$ as follows:

$$Ax = b \iff PAx = Pb \iff LUx = Pb$$

1. Rearrange the rows of b to find Pb .
2. Solve $\tilde{L}b = Pb$ using **forward substitution**.
3. Solve $Ux = \tilde{b}$ using **backward substitution**.

B.3. QR factorization

Gram-Schmidt: Given vectors $p_0, p_1, \dots, p_m$, define

$$\begin{aligned} \tilde{p}_0 &= p_0 & \tilde{p}_0^* &= \tilde{p}_0 / \|\tilde{p}_0\| \\ \tilde{p}_1 &= p_1 - (p_1 \cdot \tilde{p}_0^*)\tilde{p}_0^* & \tilde{p}_1^* &= \tilde{p}_1 / \|\tilde{p}_1\| \\ \tilde{p}_2 &= p_2 - (p_2 \cdot \tilde{p}_0^*)\tilde{p}_0^* - (p_2 \cdot \tilde{p}_1^*)\tilde{p}_1^* & \tilde{p}_2^* &= \tilde{p}_2 / \|\tilde{p}_2\| \\ \dots & \dots & \dots & \dots \\ \tilde{p}_m &= p_m - (p_m \cdot \tilde{p}_0^*)\tilde{p}_0^* - (p_m \cdot \tilde{p}_1^*)\tilde{p}_1^* - \dots - (p_m \cdot \tilde{p}_{m-1}^*)\tilde{p}_{m-1}^* & \tilde{p}_m^* &= \tilde{p}_m / \|\tilde{p}_m\| \end{aligned}$$

QR factorisation: Q is orthogonal ($Q^T Q = I$), R is upper triangular

$$(p_0, p_1, \dots, p_m) = (\tilde{p}_0^*, \tilde{p}_1^*, \dots, \tilde{p}_m^*) \begin{pmatrix} \|\tilde{p}_0\| & p_1 \cdot \tilde{p}_0^* & p_2 \cdot \tilde{p}_0^* & \cdots & p_m \cdot \tilde{p}_0^* \\ & \|\tilde{p}_1\| & p_2 \cdot \tilde{p}_1^* & \cdots & p_m \cdot \tilde{p}_1^* \\ & & \ddots & \ddots & \vdots \\ & & & \|\tilde{p}_{m-1}\| & p_m \cdot \tilde{p}_{m-1}^* \\ & & & & \|\tilde{p}_m\| \end{pmatrix}$$

C. Interpolation and Approximation

C.1. Lagrange interpolation

Finding the unique polynomial of degree $\leq n$ that passes through $\{(x_i, y_i)\}_{i=0}^n$.

- k -th **Lagrange basis polynomial:**

$$L_k(x) = \prod_{\substack{j=0 \\ j \neq k}}^n \frac{x - x_j}{x_k - x_j} = \frac{(x - x_0) \cdots (x - x_{k-1})(x - x_{k+1}) \cdots (x - x_n)}{(x_k - x_0) \cdots (x_k - x_{k-1})(x_k - x_{k+1}) \cdots (x_k - x_n)}$$

Property: $L_k(x_i) = \delta_{ik}$

Tip: If $L_k(x)$ have unknown coefficients, and asked to find interpolation nodes, expand out each $L_k(x)$ and consider ratio of coefficients of x^2 and x^3 , compare with the given expression

- **Lagrange interpolating polynomial:** $P_n(x) = \sum_{k=0}^n f(x_k)L_k(x)$

so that $P_n(x_i) = \sum_{k=0}^n f(x_k)L_k(x_i) = f(x_i)$ for $i = 0, 1, \dots, n$.

- If interpolating polynomial $P_n(x) = a_0 + a_1x + \cdots + a_nx^n$, find coefficients by solving

$$\begin{pmatrix} 1 & x_0 & \cdots & x_0^n \\ 1 & x_1 & \cdots & x_1^n \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & \cdots & x_n^n \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{pmatrix} = \begin{pmatrix} f(x_0) \\ f(x_1) \\ \vdots \\ f(x_n) \end{pmatrix}.$$

The matrix above is Vandermonde matrix. Vandermonde matrix with distinct $x_0, x_1, \dots, x_n$ are linearly independent (and have determinant $= \prod_{0 \leq i < j \leq n} (x_j - x_i)$), hence the solution exists and is unique.

C.2. Divided differences, Newton interpolation

- For $k \geq 1$, k -th **divided difference** of $f(x)$ is

$$f[x_0, x_1, \dots, x_k] = \sum_{l=0}^k f(x_l) \prod_{j=0, j \neq l}^k \frac{1}{x_l - x_j} \quad f[x_0] = f(x_0)$$

- Divided differences are leading coefficients of interpolating polynomial:

$$P_n(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + \cdots + f[x_0, x_1, \dots, x_n](x - x_0)(x - x_1) \cdots (x - x_{n-1})$$

- **Recurrence:** Let $x_0, x_1, \dots, x_n$ be $n + 1$ distinct points. For any function f ,

$$f[x_0, x_1, \dots, x_{n-1}, x_n] = \frac{f[x_1, \dots, x_n] - f[x_0, \dots, x_{n-1}]}{x_n - x_0}.$$

x	0th d.d.	1st d.d.	2nd d.d.
x_0	$f[x_0] = f(x_0)$		
		$f[x_0, x_1] = \frac{f[x_1] - f[x_0]}{x_1 - x_0}$	
x_1	$f[x_1] = f(x_1)$		$f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0}$
		$f[x_1, x_2] = \frac{f[x_2] - f[x_1]}{x_2 - x_1}$	
x_2	$f[x_2] = f(x_2)$		$f[x_1, x_2, x_3] = \frac{f[x_2, x_3] - f[x_1, x_2]}{x_3 - x_1}$
		$f[x_2, x_3] = \frac{f[x_3] - f[x_2]}{x_3 - x_2}$	
x_3	$f[x_3] = f(x_3)$		

- If interpolating nodes are equal, e.g. $x_0 = x_1$, then define d.d. as derivative:

$$f[x_0, x_0] = \lim_{x_1 \rightarrow x_0} \frac{f[x_0, x_1] - f[x_0]}{x_1 - x_0} = \lim_{x_1 \rightarrow x_0} \frac{f[x_1] - f[x_0]}{x_1 - x_0} = f'(x_0)$$

- **Linearity:**

$$\begin{aligned} f[x_0, \dots, x_n] + g[x_0, \dots, x_n] &= (f + g)[x_0, \dots, x_n] \\ \lambda f[x_0, \dots, x_n] &= (\lambda \cdot f)[x_0, \dots, x_n] \end{aligned}$$

Symmetry: For an arbitrary permutation $\sigma: \{0, \dots, n\} \rightarrow \{0, \dots, n\}$,

$$f[x_0, \dots, x_n] = f[x_{\sigma(0)}, \dots, x_{\sigma(n)}].$$

Leibniz rule:

$$(f \cdot g)[x_0, \dots, x_n] = \sum_{r=0}^n f[x_0, \dots, x_r]g[x_r, \dots, x_n].$$

C.3. Chebyshev nodes

- **Chebyshev nodes** $x_k = \cos\left(\frac{(k+1/2)\pi}{n+1}\right)$ for $k = 0, 1, \dots, n$
- **Chebyshev polynomial** of degree n is $T_n(x) = \cos(n \arccos x)$
- $T_n(x) = 2xT_{n-1}(x) - T_{n-2}(x)$ by induction, one can show that $T_n(x)$ is a polynomial
- Leading coefficient of $T_n(x)$ is 2^{n-1} . Chebyshev nodes are roots of Chebyshev polynomial, so we can factorise

$$T_{n+1}(x) = 2^n(x - x_0)(x - x_1) \cdots (x - x_n)$$

$$\text{so } g_n(x) = |2^{-n}T_{n+1}(x)| = |2^{-n} \cos((n+1) \arccos x)| \leq 2^{-n}$$

C.4. Error of Lagrange interpolation

- Let $P_n(x)$ be interpolating polynomial for $x_0, x_1, \dots, x_n$. By successive construction, interpolating polynomial for $x_0, x_1, \dots, x_n, x_{n+1}$ is

$$P_{n+1}(x) = P_n(x) + f[x_0, \dots, x_n, x_{n+1}](x - x_0) \cdots (x - x_n)$$

Since P_{n+1} matches f at x_{n+1} , we have $f(x_{n+1}) = P_{n+1}(x_{n+1})$, so

$$f(x_{n+1}) = P_n(x_{n+1}) + f[x_0, \dots, x_n, x_{n+1}](x_{n+1} - x_0) \cdots (x_{n+1} - x_n)$$

Since this holds for arbitrary x_{n+1} , replace x_{n+1} with x :

$$f(x) = P_n(x) + f[x_0, \dots, x_n, x](x - x_0) \cdots (x - x_n)$$

- Let $x_0, x_1, \dots, x_n$ be distinct. For any n th-order differentiable $f(x)$, there exists ξ s.t.

$$f[x_0, \dots, x_n] = \frac{f^{(n)}(\xi)}{n!}.$$

Proof. For $n = 1$, set $g(x) = f(x) - P_1(x)$. By Rolle, there exists ξ between x_0 and x_1 s.t.

$$f[x_0, x_1] = \frac{f(x_1) - f(x_0)}{x_1 - x_0} = f'(\xi).$$

For $n = 2$, similarly define $g(x) = f(x) - P_2(x)$. Then $g(x_0) = g(x_1) = g(x_2) = 0$. WLOG assume $x_0 < x_1 < x_2$.

By Rolle, there exists $\xi_0 \in (x_0, x_1)$ s.t. $g'(\xi_0) = 0$; there exists $\xi_1 \in (x_1, x_2)$ s.t. $g'(\xi_1) = 0$. By Rolle, there exists $\eta \in (\xi_0, \xi_1)$ s.t. $g''(\eta) = 0$.

Since $g(x) = f(x) - P_2(x)$, we have $f''(\eta) - P_2''(x) = 0$. Since $P_2''(x) = 2f[x_0, x_1, x_2]$, we conclude that $f[x_0, x_1, x_2] = \frac{1}{2}f''(\eta)$. $\square$

Note: To generalise to any positive integer n , apply Rolle $n + (n - 1) + \cdots + 1$ times.

- Let $x_0, x_1, \dots, x_n$ be distinct interpolation nodes. $P_n(x)$ be Lagrange interpolating polynomial of $f(x)$. Suppose $f(x)$ is $(n+1)$ th-order differentiable. Then for any x , there exists ξ s.t.

$$f(x) - P_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} (x-x_0)(x-x_1) \cdots (x-x_n).$$

- Corollary:

$$|f(x) - P_n(x)| \leq C(x) |(x-x_0)(x-x_1) \cdots (x-x_n)|$$

where $C(x) = \frac{1}{(n+1)!} \max_{\xi \in I_x} |f^{(n+1)}(\xi)|$, $I_x = [\min(x_0, \dots, x_n, x), \max(x_0, \dots, x_n, x)]$

Note: Differentiate $f^{(n+1)}$ to find maximum value.

Example: Suppose $f(x)$ is third-order differentiable. Show that for any distinct $a, b \in \mathbb{R}$, there exists ξ between a and b such that $f[a, b, b, b] = \frac{1}{6} f'''(\xi)$.

Consider the interpolating polynomial using the nodes a, b, b, b :

$$P_3(x) = f[a] + f[a, b](x-a) + f[a, b, b](x-a)(x-b) + f[a, b, b, b](x-a)(x-b)^2.$$

Define $g(x) = f(x) - P_3(x)$. Then $g(a) = g(b) = 0, g'(b) = 0, g''(b) = 0$.

C.5. Cubic spline interpolation

Let $S(x)$ be piecewise cubic on each sub-interval $[x_{k-1}, x_k]$. Given $f: [a, b] \rightarrow \mathbb{R}$ and a set of nodes $a = x_0 < x_1 < \dots < x_n = b$, we say that S is **cubic spline interpolant** for f if:

- $p_k(x_{k-1}) = f(x_{k-1})$ and $p_k(x_k) = f(x_k)$ for each $k = 1, \dots, n$.
- $p'_k(x_k) = p'_{k+1}(x_k)$ for each $k = 1, \dots, n-1$.
- $p''_k(x_k) = p''_{k+1}(x_k)$ for each $k = 1, \dots, n-1$.
- One of the following sets of boundary conditions is satisfied:
 - $p'_1(x_0) = f'(x_0)$ and $p'_n(x_n) = f'(x_n)$ (clamped boundary)
 - $S''(x_0) = S''(x_n) = 0$ (natural boundary)

Procedure:

- Calculate divided differences (up to 2nd d.d.)
- Solve for second-order derivatives

$$\lambda_k = \frac{x_k - x_{k-1}}{x_{k+1} - x_{k-1}}, \quad x_{-1} = x_0, \quad x_{n+1} = x_n$$

Clamped boundary conditions:

$$\begin{pmatrix} 2 & 1-\lambda_0 & & & \\ \lambda_1 & 2 & 1-\lambda_1 & & \\ & \lambda_2 & 2 & 1-\lambda_2 & \\ & & \ddots & \ddots & \ddots \\ & & & \lambda_{n-1} & 2 & 1-\lambda_{n-1} \\ & & & & \lambda_n & 2 \end{pmatrix} \begin{pmatrix} S''(x_0) \\ S''(x_1) \\ S''(x_2) \\ \vdots \\ S''(x_{n-1}) \\ S''(x_n) \end{pmatrix} = \begin{pmatrix} 6f[x_0, x_0, x_1] \\ 6f[x_0, x_1, x_2] \\ 6f[x_1, x_2, x_3] \\ \vdots \\ 6f[x_{n-2}, x_{n-1}, x_n] \\ 6f[x_{n-1}, x_n, x_n] \end{pmatrix}$$

Natural boundary conditions:

$$\begin{pmatrix} 2 & 1-\lambda_1 & & & \\ \lambda_2 & 2 & 1-\lambda_2 & & \\ & \ddots & \ddots & \ddots & \\ & & \lambda_{n-2} & 2 & 1-\lambda_{n-2} \\ & & & \lambda_{n-1} & 2 \end{pmatrix} \begin{pmatrix} S''(x_1) \\ S''(x_2) \\ \vdots \\ S''(x_{n-2}) \\ S''(x_{n-1}) \end{pmatrix} = \begin{pmatrix} 6f[x_0, x_1, x_2] \\ 6f[x_1, x_2, x_3] \\ \vdots \\ 6f[x_{n-3}, x_{n-2}, x_{n-1}] \\ 6f[x_{n-2}, x_{n-1}, x_n] \end{pmatrix}$$

- Construct the polynomials

$$p_k(x) = f(x_{k-1}) + f[x_{k-1}, x_k](x-x_{k-1}) + (x-x_{k-1})(x-x_k)(\alpha_k x + \beta_k)$$

$$\alpha_k = \frac{S''(x_k) - S''(x_{k-1})}{6(x_k - x_{k-1})}, \quad \beta_k = \frac{x_k - 2x_{k-1}}{6(x_k - x_{k-1})} S''(x_k) + \frac{2x_k - x_{k-1}}{6(x_k - x_{k-1})} S''(x_{k-1})$$

C.6. Least squares approximation

Given data points $\{(x_i, y_i)\}_{i=0}^n$, we want to find $P(x) = a_0 + a_1 x + a_2 x^2 + \dots + a_m x^m$ that minimises least squares error = sum of squares of differences between y -values on $P(x)$ and the given y -values: $E = \sum_{i=1}^n [y_i - P_m(x_i)]^2$

- For $k = 0, 1, \dots, m$, let $\mathbf{p}_k = (x_0^k, x_1^k, \dots, x_n^k)^T$
- Use Gram-Schmidt to orthonormalise $\mathbf{p}_0, \mathbf{p}_1, \dots, \mathbf{p}_n$.
- Obtain a QR factorisation of

$$X = \begin{pmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^m \\ 1 & x_1 & x_1^2 & \cdots & x_1^m \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^m \end{pmatrix} = (\mathbf{p}_0, \mathbf{p}_1, \dots, \mathbf{p}_n)$$

- Let $\mathbf{b} = Q^T \mathbf{y}$. The point on the hyperplane closest to $\mathbf{y}$ is

$$\mathbf{y}^* = b_0 \mathbf{p}_0^* + b_1 \mathbf{p}_1^* + \dots + b_m \mathbf{p}_m^*$$

The shortest distance is $\|\mathbf{y} - \mathbf{y}^*\|$

- To find coefficients of $P(x)$, solve $\mathbf{R} \mathbf{a} = Q^T \mathbf{y}$ for $\mathbf{a}$.

D. Numerical Integration

D.1. Newton-Cotes formulas

- Trapezoidal rule:**

$$\int_a^b f(x) dx \approx \frac{b-a}{2} [f(a) + f(b)]$$

Error term: $-\frac{1}{12}(b-a)^3 f''(\xi)$, differentiate again to find $\max_{\xi \in [a,b]} |f'''(\xi)|$

- Simpson's rule:**

$$\int_a^b f(x) dx \approx \frac{b-a}{6} \left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right]$$

Error term: $-\frac{1}{90}\left(\frac{b-a}{2}\right)^5 f^{(4)}(\xi)$

- In general, error term is

$$\int_a^b f(x) dx - (\text{numerical integration}) = C(b-a)^{k+1} f^{(k)}(\xi)$$

Degree of accuracy is k , meaning numerical integration formula is exact for all polynomials of $\deg \leq k$, inexact for $\deg = k+1$

- If $\deg f \leq k-1$, then $f^{(k)}(x) = 0$, so numerical integration is exact.
- If $\deg f = k$, then error is non-zero constant.

Take $a = -1, b = 1$, test using polynomials $f(x) = 1, x, x^2, \dots$ until numerical integration is inexact. To find the constant C , choose $a = -1, b = 1$ and $f(x)$ as above.

- Closed Newton-Cotes formula:** $x_0 = a, x_n = b, h = \frac{b-a}{n}$

$$\int_a^b f(x) dx \approx (b-a) \sum_{i=0}^n c_i f(x_i) = h \sum_{i=0}^n H_{n,i} f(x_i)$$

$$H_{n,i} = \frac{(-1)^{n-i}}{i!(n-i)!} \int_0^n s(s-1) \cdots (s-i+1)(s-i-1) \cdots (s-n) ds$$

- Special case: **Simpson's 3/8 rule:**

$$\int_a^b f(x) dx \approx \frac{b-a}{3} \left[\frac{3}{8} f(a) + \frac{9}{8} f\left(\frac{2a+b}{3}\right) + \frac{9}{8} f\left(\frac{a+2b}{3}\right) + \frac{3}{8} f(b) \right]$$

- Error term: There exists $\xi \in (a, b)$ such that

$$\int_a^b f(x) dx = \sum_{i=0}^n a_i f(x_i) + \frac{h^{n+2} f^{(n+2)}(\xi)}{(n+2)!} \int_0^n t^2(t-1) \cdots (t-n) dt$$

if n is even and $f \in C^{n+2}[a, b]$, and

$$\int_a^b f(x) dx = \sum_{i=0}^n a_i f(x_i) + \frac{h^{n+2} f^{(n+1)}(\xi)}{(n+1)!} \int_0^n t(t-1) \cdots (t-n) dt$$

if n is odd and $f \in C^{n+1}[a, b]$.

- Open Newton-Cotes formula:** use interior nodes $x_i = a + ih, h = \frac{b-a}{n+2}, i = 1, \dots, n+1$

$$\int_a^b f(x) dx \approx (b-a) \sum_{i=1}^{n+1} c_i f(x_i) = h \sum_{i=1}^{n+1} H_{n,i} f(x_i)$$

$$H_{n,i} = \frac{(-1)^{n+1-i}}{(i-1)!(n+1-i)!} \int_0^{n+1} s(s-1) \cdots (s-i+2)(s-i) \cdots (s-(n+1)) ds$$

- $n = 0$: Midpoint rule

$$\int_a^b f(x) dx \approx 2hf\left(\frac{a+b}{2}\right) + \frac{h^3}{3} f''(\xi)$$

- $n = 1$:

$$\int_a^b f(x) dx \approx \frac{3h}{2} \left[f\left(\frac{2a+b}{3}\right) + f\left(\frac{a+2b}{3}\right) \right] + \frac{3h^3}{4} f''(\xi)$$

- $n = 2$:

$$\int_a^b f(x) dx \approx \frac{4h}{3} \left[2f\left(\frac{3a+b}{4}\right) - f\left(\frac{a+b}{2}\right) + 2f\left(\frac{a+3b}{4}\right) \right] + \frac{14h^4}{45} f^{(4)}(\xi)$$

- $n = 3$:

$$\int_a^b f(x) dx \approx \frac{5h}{24} \left[11f\left(\frac{4a+b}{5}\right) + f\left(\frac{3a+2b}{5}\right) + f\left(\frac{2a+3b}{5}\right) + 11f\left(\frac{a+4b}{5}\right) \right] + \frac{95h^5}{144} f^{(4)}(\xi)$$

- Error term: $x_{-1} = a, x_{n+1} = b$. There exists $\xi \in (a, b)$ for which

$$\int_a^b f(x) dx = \sum_{i=0}^n a_i f(x_i) + \frac{h^{n+3} f^{(n+2)}(\xi)}{(n+2)!} \int_{-1}^{n+1} t^2(t-1) \cdots (t-n) dt$$

if n is even and $f \in C^{n+2}[a, b]$, and

$$\int_a^b f(x) dx = \sum_{i=0}^n a_i f(x_i) + \frac{h^{n+2} f^{(n+1)}(\xi)}{(n+1)!} \int_{-1}^{n+1} t(t-1) \cdots (t-n) dt$$

if n is odd and $f \in C^{n+1}[a, b]$.

Example: Find constants c_0, c_1, x_1 so that $\int_{-1}^1 f(x) dx = c_0 f(-1) + c_1 f(x_1)$ has highest degree of precision. Test the formula with different polynomials. Since there are 3 unknowns, we write down 3 equations.

- If $f(x) = 1$, then $2 = c_0 + c_1$.
- If $f(x) = x$, then $0 = -c_0 + c_1 x_1$.
- If $f(x) = x^2$, then $\frac{2}{3} = c_0 + c_1 x_1^2$.

Solving for the unknowns gives $c_0 = \frac{1}{2}, c_1 = \frac{3}{2}, x_1 = \frac{1}{3}$. Hence, $\int_{-1}^2 f(x) dx = \frac{1}{2} f(-1) + \frac{3}{2} \left(\frac{1}{3}\right)$ has highest degree of precision of 2, by testing with $f(x) = x^3$.

D.2. Composite integration rules

- Composite trapezoidal rule:**

$$\int_a^b f(x) dx \approx \sum_{k=1}^n \frac{x_k - x_{k-1}}{2} [f(x_{k-1}) + f(x_k)]$$

If interpolating nodes equally spaced by $h = \frac{b-a}{n}$,

$$\int_a^b f(x) dx \approx h \left[\frac{1}{2} f(x_0) + \sum_{k=1}^{n-1} f(x_k) + \frac{1}{2} f(x_n) \right]$$

- Error term: $-\frac{b-a}{12} h^2 f''(\xi)$, bounded by $\frac{b-a}{12} h^2 \max_{\xi \in [a,b]} |f''(\xi)| = \frac{(b-a)^3}{12n^2} \max_{\xi \in [a,b]} |f''(\xi)|$

Order of convergence: 2

Note: Since need to differentiate f'' again to find max, calculate required derivatives beforehand

- Composite Simpson's rule:**

$$\int_a^b f(x) dx \approx \frac{h}{3} \left[f(a) + f(b) + 2 \sum_{k=1}^{n/2-1} f(x_{2k}) + 4 \sum_{k=1}^{n/2} f(x_{2k-1}) \right]$$

Note: Number of nodes must be odd (n must be even), since we need to use the midpoint of each subinterval $[x_0, x_2], [x_2, x_4], \dots, [x_{n-2}, x_n]$.

- Error term: $-\frac{b-a}{180} h^4 f^{(4)}(\xi)$, bounded by $\frac{b-a}{180} h^4 \max_{\xi \in [a,b]} |f^{(4)}(\xi)| = \frac{(b-a)^5}{180n^4} \max_{\xi \in [a,b]} |f^{(4)}(\xi)|$

Order of convergence: 4

Miscellaneous

Calculus

- IVT:** Let f cont on $[a, b]$. Then for any u between $f(a)$ and $f(b)$, $\exists c \in (a, b)$ s.t. $f(c) = u$.
- Rolle:** Let f cont on $[a, b]$, diff on (a, b) and $f(a) = f(b)$. Then $\exists c \in (a, b)$ s.t. $f'(c) = 0$
- MVT:** Let f cont on $[a, b]$, diff on (a, b) . Then $\exists c \in (a, b)$ s.t. $f'(c) = \frac{f(b)-f(a)}{b-a}$.